MIS 600 Database Management Systems

Chapter 5 Assignment

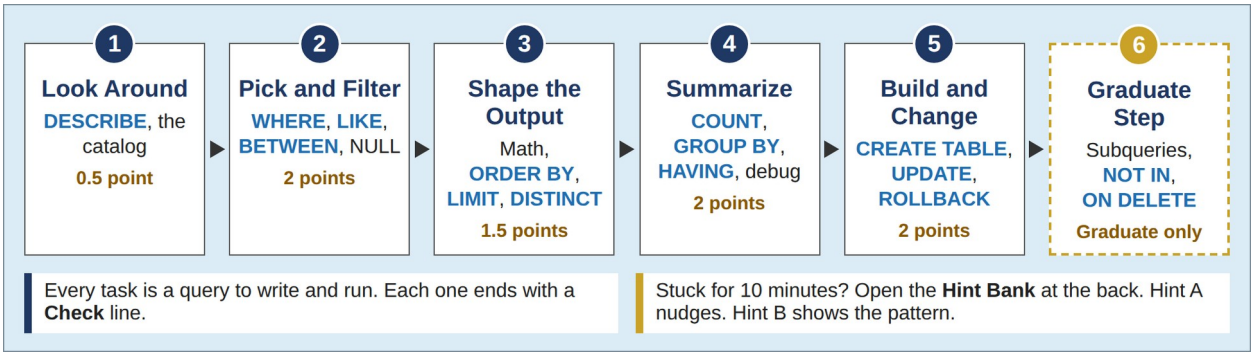
SQL Practice at Pine Valley

Name: _____ Undergraduate / Graduate (circle one)

Due: _____ Points: 8

This assignment is pure SQL practice on what we covered in Chapter 5 and in the SQL drills. Every task is a request from the Pine Valley sales manager, and you answer each one with a query. The parts follow the same rounds as the drills, so work in order.

You may work in **MySQL** or in **SQL Server**. Pick one and stay with it. Most of your SQL runs the same in both. Where SQL Server differs, a purple line that starts with **SQL Server:** tells you, right under the task.



Your roadmap: five parts for everyone, plus one for graduate students

How this assignment works

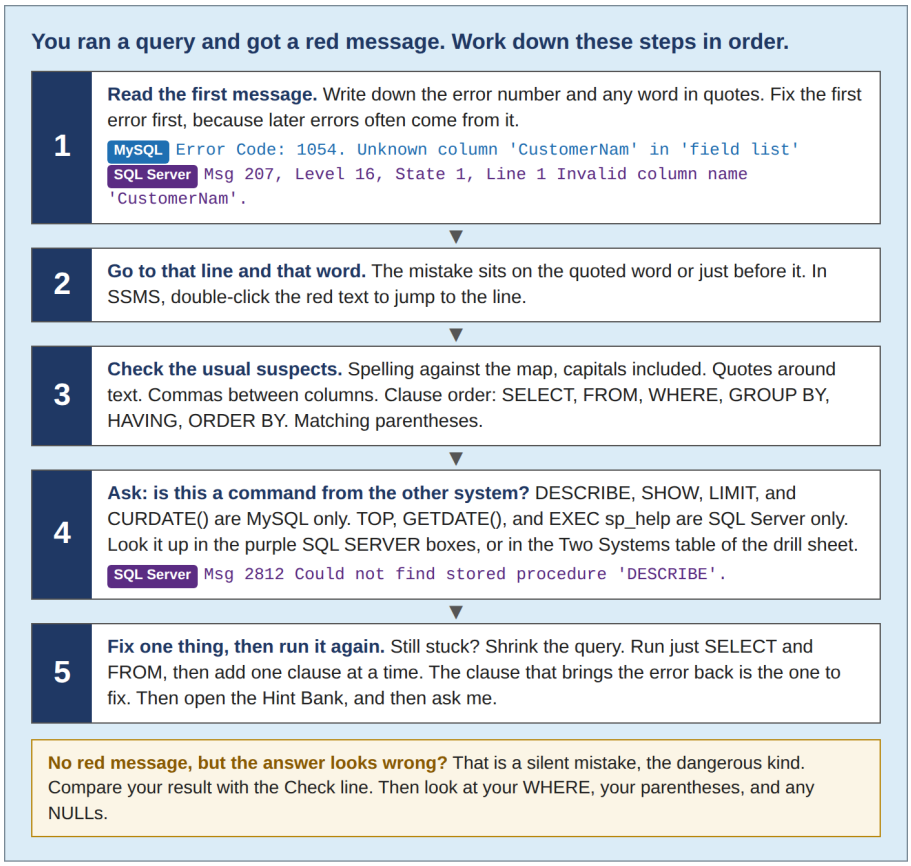
- Set up your system with the table on the next page.
- Keep all of your queries in one place, in task order. Part 5 changes data, and the tasks after it expect those changes.
- For each task, submit your SQL typed as text and a screenshot of the result, all in one Word document on Blackboard. Write MySQL or SQL Server at the top.
- Got an error? Use the five steps on the next page before you open a hint.
- Every task ends with a **Check** line. If your result does not match it, fix the query before you move on.
- Stuck for 10 minutes? Open the **Hint Bank** at the back. Try Hint A first. Hint B shows the pattern.
- Generative AI is not allowed on this assignment. This is the course policy.

Getting Set Up

Pick your system and follow its column. When a task shows two boxes, one tagged **MySQL** and one tagged **SQL Server**, run only the one for your system. A purple line that starts with **SQL Server:** tells you what changes in SQL Server.

	MySQL in DB Fiddle	SQL Server in SSMS
Where you work	db-fiddle.com. Choose **MySQL v8.0** in the database menu.	Your own database in **SQL Server Management Studio**. Right-click Databases, choose New Database, and name it PVFC.
Load Pine Valley	Paste the **PVFC_SQL** script into the Schema SQL panel. Stop before `-- SAMPLE QUERIES`. Click Build Schema.	Open **PVFC_SQL_SQLServer.sql**. Pick PVFC in the database drop-down on the toolbar. Press F5.
Run a query	Type in the Query SQL panel and click Run.	Click New Query. Highlight the statement you want and press F5.
Start over	Every Run rebuilds the database, so you cannot break anything.	Your changes stay. To start over, run `DROP TABLE IF EXISTS Review_T;` and then the script again.
Errors	Red text under the query panel, with an error code.	Messages tab: red Msg number and Line. Double-click it to jump to the line.

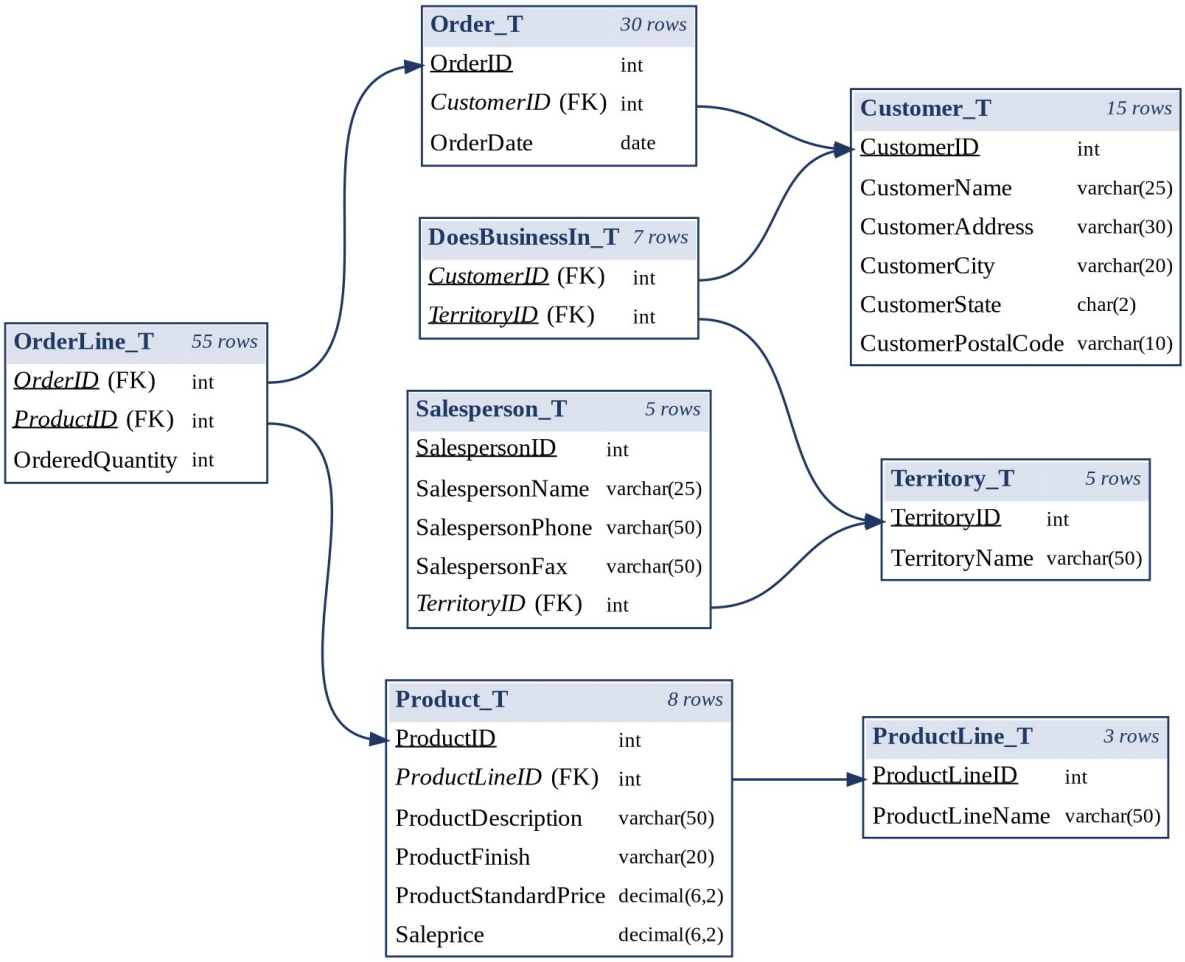
When you get an error



Five steps for any error message

The Map: Keep This Page Open

An underlined column is part of the primary key. An italic column marked (FK) is a foreign key, and its arrow points to the table it references. Spell every table and column exactly as the map shows.



Pine Valley Furniture: tables, columns, keys, and connections

Every SELECT follows this shape. Words in brackets are optional, and the clauses always go in this order.

```
SELECT [ALL/DISTINCT] column_list
FROM table_list
[WHERE conditional expression]
[GROUP BY group_by_column_list]
[HAVING conditional expression]
[ORDER BY order_by_column_list]
```

The shape of a SELECT statement

Part 1: Look Around

Task 1.1 Read two table designs (0.25 point)

Ticket: *"Show me how Order_T is built, and show me the code that created Product_T."*

Check: DESCRIBE gives 3 rows, and OrderDate defaults to curdate(). The Product_T code includes a CHECK rule named ProductFinish_CK.

SQL Server: Use EXEC sp_help for the design. For the code, right-click the table in Object Explorer and choose Script Table as, CREATE To, New Query Editor Window.

SQL Server check: sp_help lists 3 columns and a DEFAULT on OrderDate. The Product_T script shows ProductFinish_CK.

Task 1.2 Ask the catalog

Ticket: *"List every foreign key in our schema: the table it sits in, its name, and the table it points to. Sort by table name."*

Check: 18 rows. The first table is DoesBusinessIn_T.

SQL Server: Read sys.foreign_keys. OBJECT_NAME() turns an object number into a table name. The check is the same.

Part 2: Pick and Filter

Each task answers one question with SELECT, FROM, and WHERE. Start with the worked example. After that, each task gives you a little less help.

WORKED EXAMPLE	**Ticket:** "Which of our customers are in Florida? List them by name." Type this query in DB Fiddle or SSMS and run it before you start Task 2.1. Each clause answers one small question.

`SELECT CustomerName, CustomerCity`	**Which columns** to show, in this order
`FROM Customer_T`	**Which table**, spelled exactly as on the map
`WHERE CustomerState = 'FL'`	**Which rows** to keep. Text goes in single quotes.
`ORDER BY CustomerName;`	**What order** to show them in. The semicolon ends the statement.

The result: 3 of the 15 customers

CustomerName	CustomerCity
Contemporary Casuals	Gainesville
M and H Casual Furniture	Clearwater
Seminole Interiors	Seminole

A worked example: each clause answers one small question

Task 2.1 Fill in the blanks (0.4 point)

HELP LEVEL: fill in the blanks

Ticket: *"List the ID, description, and standard price of every product that costs less than $300."*

```
SELECT ProductID, _________________, ProductStandardPrice
FROM _________________
WHERE _________________ < 300;
```

Check: 3 rows. The most expensive of them costs 250.00.

Task 2.2 Use a pattern (0.4 point)

HELP LEVEL: use the pattern

Ticket: *"Which orders did we receive in October 2024? Show the order ID and the date, oldest first."*

Pattern: WHERE column BETWEEN low AND high

Check: 8 rows. The first is order 1021.

Task 2.3 On your own (0.4 point)

HELP LEVEL: on your own

Ticket: *"Which customers have the word Furniture or the word Furnishings anywhere in their name?"*

Check: 8 rows.

Task 2.4 Mix AND with OR (0.4 point)

HELP LEVEL: on your own

Ticket: *"Show every Cherry or Walnut product that costs more than $250: description, finish, and price."*

Check: 1 row. If you get 2, look at your parentheses.

Task 2.5 Find the missing values (0.4 point)

HELP LEVEL: on your own

Ticket: *"Which employees have no supervisor on file? Show the employee ID and name."*

Check: 1 row. If you get 0 rows, look at how you test for NULL.

Part 3: Shape the Output (1.5 points)

Task 3.1 Calculate a column (0.5 point)

Ticket: *"Show each product's description, its standard price, and its price after a 15 percent discount. Call the new column DiscountPrice. Most expensive first."*

Check: 8 rows. The first row is the Dining Table: 800.00 and 680.0000.

Task 3.2 Keep only the top rows (0.5 point)

Ticket: *"Show our three cheapest products with their prices, cheapest first."*

Check: 3 rows. The first is the End Table at 175.00.

SQL Server: There is no LIMIT. Put TOP right after SELECT. The check is the same.

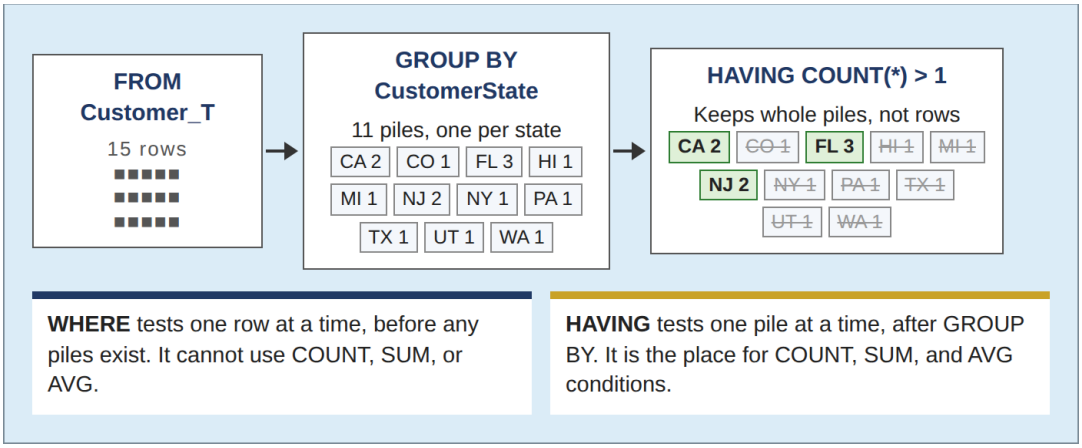
Task 3.3 Remove duplicates (0.5 point)

Ticket: *"Which wood finishes do we sell? List each finish once, in alphabetical order."*

Check: 5 rows, from Cherry to White Ash.

Part 4: Summarize (2 points)

These tasks turn many rows into a few answers. Keep the figure in mind: WHERE and HAVING filter different things.



WHERE keeps rows before grouping. HAVING keeps whole groups after it. (Textbook example)

Task 4.1 Total each group (0.4 point)

Ticket: *"How many units of each product have we sold? Show ProductID and TotalUnits, best seller first."*

Check: 8 rows. The best seller is product 2, with 61 units.

Task 4.2 Filter the groups (0.4 point)

Ticket: *"Which customers have placed more than 3 orders? Show CustomerID and NumOrders."*

Check: 3 rows. The top customer has 7 orders.

Task 4.3 Average and round (0.4 point)

Ticket: *"Which product lines have an average standard price above $400? Show ProductLineID and the average, rounded to 2 decimals."*

Check: 2 rows. One of them averages 450.00.

SQL Server: You see 450.000000. That counts as a match.

Task 4.4 Group on a function (0.4 point)

Ticket: *"For each year, how many orders did we get, and how many different customers placed them? Call the columns OrderYear, NumOrders, and NumCustomers."*

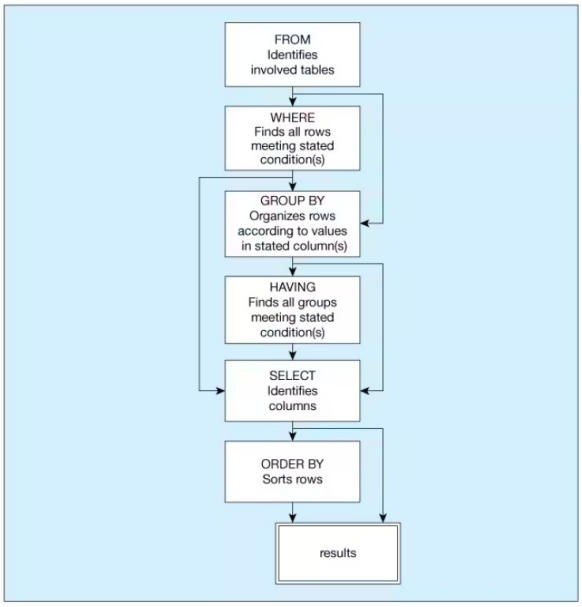
Check: 3 rows, 10 orders each year. In 2024, 9 different customers ordered.

Task 4.5 Fix a broken query (0.4 point)

This query should list each finish with more than one product, largest group first. It has three mistakes. The database reports one at a time, so fix the first error, run again, and repeat. Submit the fixed query.

```
SELECT ProductFinish COUNT(*) AS NumProducts
FROM Products_T
WHERE COUNT(*) > 1
GROUP BY ProductFinish
ORDER BY NumProducts DESC;
```

Check: 2 rows: Natural Ash 3 and Cherry 2.



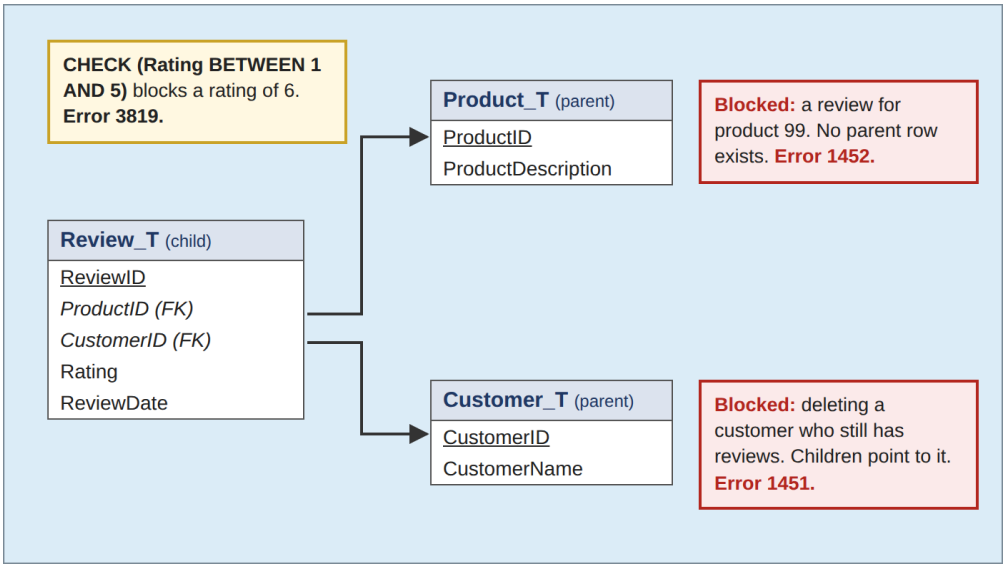
The order SQL runs the clauses (Figure 5-10)

Part 5: Build and Change (2 points)

Pine Valley wants customers to rate products from 1 to 5. You build the table that holds the ratings, add data, and change data the safe way.

DB FIDDLE	A failed statement can stop the ones after it. When a Part 5 statement fails on purpose, take a screenshot of the error, then put -- at the start of that statement so the rest can run.

SQL Server: Your changes stay in your database. Highlight one statement at a time and press F5, so each error gets its own screenshot. To start over, run DROP TABLE IF EXISTS Review_T; and then the PVFC_SQL_SQLServer script again.



Review_T is a child of two parents. Its keys and its CHECK rule block bad data.

Task 5.1 Create the table (0.5 point)

Ticket: *"Create Review_T with the columns and rules below. Name every constraint as shown."*

Column	Data type	Rule
ReviewID	INT	NOT NULL. Primary key named Review_PK.
ProductID	INT	NOT NULL. Foreign key to Product_T, named Review_FK1.
CustomerID	INT	NOT NULL. Foreign key to Customer_T, named Review_FK2.
Rating	INT	NOT NULL. CHECK rule named Review_CK1: from 1 to 5.
ReviewDate	DATE	Defaults to today. MySQL: DEFAULT (CURRENT_DATE) SQL Server: DEFAULT (CAST(GETDATE() AS date))

Then look at your table:

MySQL
```
DESCRIBE Review_T;
```

SQL Server
```
EXEC sp_help 'Review_T';
```

Check: 5 rows. ReviewID shows PRI, and ReviewDate defaults to curdate().

SQL Server check: sp_help lists 5 columns, then Review_PK, Review_FK1, Review_FK2, Review_CK1, and a DEFAULT on ReviewDate.

Task 5.2 Insert rows and test the rules (0.5 point)

Ticket: *"Add these three reviews, then show the table. Review 3 has no date, so let the default fill it."*

ReviewID	ProductID	CustomerID	Rating	ReviewDate
1	7	1	5	2026-09-20
2	2	3	4	2026-09-21
3	7	9	3	(leave it out)

Check: 3 rows. Review 3 shows today's date.

Now write and run these three statements. Each one must fail.

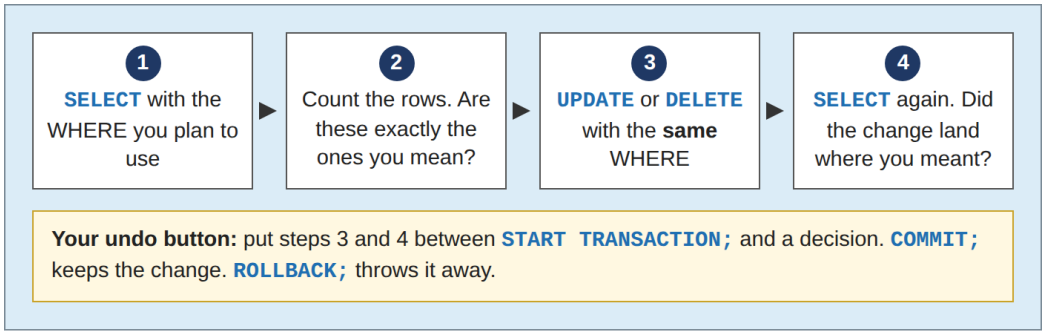
- Insert review 4 for product 99, by customer 1, rated 5.
- Insert review 4 for product 5, by customer 2, rated 6.
- Delete customer 9 from Customer_T.

Check: Errors 1452, 3819, and 1451, in that order.

SQL Server check: Msg 547 three times. The messages name Review_FK1, then Review_CK1, then Review_FK2.

Task 5.3 Change data the safe way (0.5 point)

Ticket: *"Raise the standard price of every Natural Ash product by 5 percent."*



The safe-change loop: look first, change second, check third

Write three statements: a SELECT with your WHERE, the UPDATE with the same WHERE, and the same SELECT again.

Check: 3 rows before and after. The Dining Table goes from 800.00 to 840.00.

Task 5.4 Undo with a transaction (0.5 point)

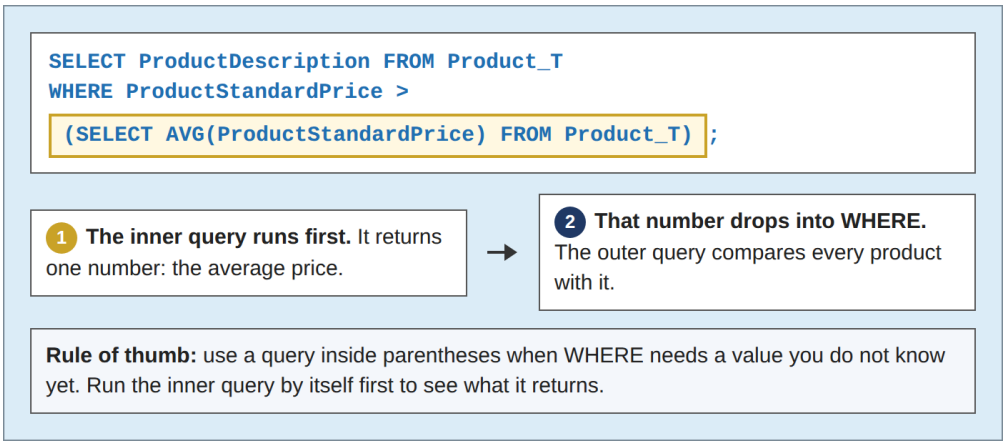
Ticket: *"Delete review 2, count the reviews, then take the delete back and count again."*

Check: 2, then 3.

SQL Server: Start with BEGIN TRANSACTION; in place of START TRANSACTION;. COMMIT and ROLLBACK are the same.

Part 6: Graduate Step (required for graduate students)

These three tasks are required for graduate students. A missing Part 6 costs a graduate student 1 point. Do them in order, after Part 5.



A query inside a query: the inner one runs first

Task 6.1 Compare with the average (graduate)

Ticket: *"Which products cost more than the average standard price? Show description and price. Do not type the average yourself."*

Check: 3 rows.

Task 6.2 Find what is missing (graduate)

Ticket: *"Which customers have never placed an order? Show CustomerID and CustomerName."*

Check: 6 rows.

Task 6.3 Change the delete rule (graduate)

Ticket: *"When we delete a customer, delete that customer's reviews too. Then test it with customer 7."*

- Replace Review_FK2 with a version that uses ON DELETE CASCADE.
- Add review 4 for product 1, by customer 7, rated 4. Count the reviews.
- Delete customer 7. Count the reviews again.

Check: 4, then 3.

SQL Server: Remove a foreign key with DROP CONSTRAINT, not DROP FOREIGN KEY.

Before You Submit

- ☐ The top of my document says MySQL or SQL Server.
- ☐ Every task has my SQL typed as text and a screenshot of the result.
- ☐ Every result matches its Check line.
- ☐ Part 5 includes a screenshot of each of the three errors.
- ☐ Graduate students: Part 6 is complete.

Hint Bank

Try each task for 10 minutes, and work through the five error steps, before you look here. Read Hint A first. Read Hint B only if Hint A was not enough.

Task	Hint A: a nudge	Hint B: the pattern
1.1	Drill 2 and Drill 3 used these two commands.	**MySQL** DESCRIBE Order_T; SHOW CREATE TABLE Product_T; **SQL Server** EXEC sp_help 'Order_T'; Script Table as, CREATE To
1.2	The catalog holds every rule. Drill 4 read it.	**MySQL** FROM information_schema.KEY_COLUMN_USAGE WHERE TABLE_SCHEMA = DATABASE() AND REFERENCED_TABLE_NAME IS NOT NULL **SQL Server** SELECT OBJECT_NAME(parent_object_id), name, OBJECT_NAME(referenced_object_id) FROM sys.foreign_keys
2.1	The blanks are a column, a table, and a column.	Find the description and price columns in the Product_T box on the map.
2.2	BETWEEN takes the low value first. Dates go in quotes.	WHERE OrderDate BETWEEN '2024-10-01' AND '2024-10-31', then ORDER BY OrderDate.
2.3	% matches any run of characters, even none.	WHERE CustomerName LIKE '%Furniture%' OR CustomerName LIKE '%Furnishings%'
2.4	SQL runs AND before OR.	WHERE (condition OR condition) AND condition. IN (...) also works for the two finishes.
2.5	NULL is never equal to anything, even NULL.	WHERE EmployeeSupervisor IS NULL
3.1	Math on a column works row by row.	ProductStandardPrice * 0.85 AS DiscountPrice, then ORDER BY ProductStandardPrice DESC.
3.2	Sort first, then keep the first rows.	**MySQL** ORDER BY ProductStandardPrice LIMIT 3 **SQL Server** SELECT TOP 3 ... ORDER BY ProductStandardPrice
3.3	One keyword goes right after SELECT.	SELECT DISTINCT ProductFinish ... ORDER BY ProductFinish
4.1	Which table records how many units were ordered?	SUM(OrderedQuantity) AS TotalUnits, GROUP BY ProductID, ORDER BY TotalUnits DESC
4.2	Each row of Order_T is one order.	GROUP BY CustomerID HAVING COUNT(*) > 3
4.3	ROUND(value, 2) rounds to cents.	GROUP BY ProductLineID HAVING AVG(ProductStandardPrice) > 400
4.4	YEAR(OrderDate) pulls the year out of a date.	COUNT(*) and COUNT(DISTINCT CustomerID), GROUP BY YEAR(OrderDate)

Task	Hint A: a nudge	Hint B: the pattern
4.5	Fix only the first error the database reports, then run again.	One mistake is punctuation, one is a table name, and one is a clause.
5.1	Copy the shape of CREATE TABLE Order_T in the script: columns first, then CONSTRAINT lines.	CONSTRAINT Review_FK1 FOREIGN KEY (ProductID) REFERENCES Product_T(ProductID) and CONSTRAINT Review_CK1 CHECK (Rating BETWEEN 1 AND 5)
5.2	List the columns you fill. Leave ReviewDate out of the list for review 3.	INSERT INTO Review_T (ReviewID, ProductID, CustomerID, Rating) VALUES (3, 7, 9, 3);
5.3	Use the same WHERE in all three statements.	SET ProductStandardPrice = ProductStandardPrice * 1.05
5.4	The transaction must start before the DELETE.	**MySQL** START TRANSACTION; DELETE ...; SELECT COUNT(*) ...; ROLLBACK; SELECT COUNT(*) ...; **SQL Server** BEGIN TRANSACTION; DELETE ...; SELECT COUNT(*) ...; ROLLBACK; SELECT COUNT(*) ...;
6.1	Run the inner query alone first.	WHERE ProductStandardPrice > (SELECT AVG(ProductStandardPrice) FROM Product_T)
6.2	A customer who never ordered is not in Order_T.	WHERE CustomerID NOT IN (SELECT CustomerID FROM Order_T)
6.3	Drop the old foreign key, then add the new one.	**MySQL** ALTER TABLE Review_T DROP FOREIGN KEY Review_FK2; then ADD CONSTRAINT Review_FK2 FOREIGN KEY ... ON DELETE CASCADE **SQL Server** ALTER TABLE Review_T DROP CONSTRAINT Review_FK2; then ADD CONSTRAINT Review_FK2 FOREIGN KEY ... ON DELETE CASCADE